RESEARCH PROJECT PROPOSAL

SAE-Guided Modality-Specific Unlearning in Vision-Language Models

Base Model LLaVA-1.5-7B (HuggingFace)	**Vision Encoder** CLIP ViT-L/14-336px (OpenAI)
Vision SAE ExplainableML/sae-for-vlm (NeurIPS 2025)	**Language SAE** EvolvingLMMs-Lab/multimodal-sae (ICCV 2025)
Compute Budget Kaggle Free Tier (2x T4 GPUs, ~30 GPU hrs/account)	**Forget Aspects** 3 visual concepts (see Section 2)

Research Goal: Investigate whether SAE-based feature steering (not zero-ablation) in the visual encoder of a VLM can achieve meaningful unlearning of a visual concept, and characterize how vision-only unlearning compares against language-only and joint unlearning — in terms of forgetting quality, cross-modal leakage, and model utility.

1. Project Overview & Research Questions

This project investigates a new approach to machine unlearning in Vision-Language Models (VLMs): instead of gradient-based weight modification, we use Sparse Autoencoders (SAEs) to identify and surgically steer concept-specific features in either the visual encoder, the language model, or both. The central contribution is the empirical characterization of modality-specific unlearning — what happens when you forget a concept in vision but not in language, and vice versa?

Core Research Questions

- RQ1: Can SAE feature steering in CLIP's vision encoder cause a VLM (LLaVA) to 'forget' a visual concept, as measured by VQA performance and cross-modal probing?
- RQ2: How does vision-only SAE unlearning compare against language-only SAE unlearning and joint (both modalities) gradient unlearning?
- RQ3: Does knowledge of a forgotten visual concept 'leak' through the language modality — i.e., can the LM still answer text-only questions about the unlearned concept even after vision unlearning?

- RQ4: Does zero-ablation (hard removal of features) perform worse than negative clamping / direction steering for VLM unlearning? (Confirming prior LLM results in the multimodal domain)
- RQ5 (Fun/Bonus): If we blur or mask the identifying region of an image (face, distinctive feature), does the unlearned model still identify the concept through remaining cues?

2. Model Stack & Tool Selection

2.1 Target Model: LLaVA-1.5-7B

The primary model for this project is LLaVA-1.5-7B (llava-hf/llava-1.5-7b-hf on HuggingFace). This choice is deliberate and well-justified:

- Architecture clarity: It has a clean three-component design — CLIP ViT-L/14-336px vision encoder + 2-layer MLP projector + Vicuna-7B language model. Each component is independently accessible.
- Kaggle compatibility: Runs at ~6GB VRAM with 4-bit quantization (BitsAndBytes). At fp16 it uses ~14GB, which fits on Kaggle's dual T4 (2x 16GB = 32GB). For inference-only evaluation tasks, 4-bit quantization is sufficient.
- SAE coverage: The ExplainableML/sae-for-vlm paper (NeurIPS 2025) specifically tested their CLIP SAEs on LLaVA, demonstrating that visual SAE interventions directly steer LLaVA's text outputs. This is the exact property needed for this project.
- Community support: Extensive documentation, HuggingFace integration, and active community make debugging much faster.

2.2 SAE Resources

SAE	Covers	Source	Status
Vision SAE (CLIP)	CLIP ViT-L/14 vision encoder activations	ExplainableML/sae-for-vlm (NeurIPS 2025)	Pretrained checkpoints available on GitHub
Language SAE (LLaVA)	LLaVA-NeXT LLaMA-8B language layers	EvolvingLMMs-Lab/multimodal-sae (ICCV 2025)	Pretrained + training code available
Alternative Vision SAE	CLIP ViT (Matryoshka/hierarchical)	WolodjaZ/MSAE (ICML 2025)	Backup if primary SAE doesn't have needed features

Important note on the language SAE: The ICCV 2025 multimodal-sae repo is built on LLaVA-NeXT-LLaMA-8B, not LLaVA-1.5-7B. You have two options: (a) use LLaVA-NeXT as the primary model instead (also Kaggle-compatible), or (b) use the training code to train a small SAE on LLaVA-1.5's language layers yourself. Option (b) adds ~4-6 hours of Kaggle compute but gives complete control. Task 2 (see below) will determine which is viable.

3. Forget Aspects: What to Unlearn

We will select three visual 'aspects' (concepts) to unlearn. Each is chosen because: (1) SAE features for it have been identified in prior work, (2) it enables clear evaluation, and (3) it supports the fun knowledge-leakage experiment. We define three tiers by difficulty and research interest:

Aspect A (Primary): Animal Stripe Texture — Zebra

- What it is: The visual feature of 'black and white stripes' / 'zebra stripe pattern'. CLIP SAEs are known to learn texture features very cleanly. The Joseph et al. (2025) steering paper specifically identified texture-style features in CLIP.
- Why it works: Zebras are a clear ImageNet class. You can evaluate the model on: (1) Show image of zebra → does model identify it? (2) Show image of zebra with stripes blurred → does model still identify it? (3) Text-only Q: 'What does a zebra look like?' → does LM still know?
- Knowledge leakage test: Blur/mask the stripes in the image using simple PIL operations (Gaussian blur over the zebra body). A truly unlearned vision model should no longer recognize 'zebra' from a partially obscured image. The LM, if not unlearned, may still answer correctly from text memory.
- Dataset: ImageNet validation set (zebra class: n02391049), ~50 images. Available as 'imagenet-1k' on HuggingFace. No need to download the full ImageNet — just the zebra class.
- VQA questions: 'What animal is in this image?', 'Describe the texture/pattern of this animal.', 'Is this a horse or a zebra?'

Aspect B (Secondary): Color Attribute — Redness

- What it is: The visual concept of 'red' as a dominant color attribute. Color features are among the most reliably isolated in CLIP SAEs (the MSAE paper showed clear color features).
- Why it works: Red objects span many categories (fire trucks, apples, red cars, stop signs). Unlearning 'redness' should affect ALL of these, giving you a broad forget set and a clear retain set of non-red objects.
- Knowledge leakage test: Show a grayscale image of a fire truck → does the unlearned model still identify it as 'red'? Replace the image color with green and ask 'What color is this truck?'
- Dataset: Curated from COCO validation set — filter images where the dominant object is red. ~100 images. Retain set: images dominated by blue or green objects.
- VQA questions: 'What color is the dominant object?', 'Describe the colors in this image.', 'Is this vehicle red or blue?'

Aspect C (Bonus / Noun aspect): Entity — Fire Truck

- What it is: The visual concept of 'fire truck' as a specific vehicle entity. Fire trucks are visually distinctive (red + ladder + specific shape). They appear in COCO and ImageNet.
- Why it works best for the fun experiment: Fire trucks have highly distinctive visual features (ladders, hoses, bright red color, emergency lights). You can: (1) Show fire truck with the iconic red body photoshopped to white/grey → does unlearned model still identify it? (2) Show fire truck with the ladder removed (cropped/blurred) → does model identify it via color alone?
- Dataset: COCO validation set, 'fire hydrant' and 'truck' categories plus manually curated fire truck images (~50 images).
- VQA questions: 'What type of emergency vehicle is this?', 'What is this vehicle used for?', 'Is this a bus or an emergency vehicle?'

Recommendation: Start with Aspect A (Zebra Stripes) as it is most tractable — zebra is a well-known ImageNet class with clear SAE features, a clean forget/retain split, and excellent potential for the blurring experiment. Aspect B can be pursued in parallel if compute allows. Aspect C is the fun bonus.

4. Evaluation Framework

Evaluation in machine unlearning must simultaneously measure: how well the model forgets, how well it retains everything else, and whether cross-modal leakage occurred. We adapt established LLM unlearning metrics (MUSE, TOFU) for the multimodal setting.

4.1 Dataset Splits

Split	Definition	Used For
Forget Set (D_x)	~50-100 images of the target concept (e.g., zebra) + paired VQA questions	Measuring forgetting — model should perform poorly here after unlearning
Retain Set (D_r)	~200-300 images of similar but non-forgotten concepts (e.g., horses, donkeys)	Measuring utility — model should perform well here after unlearning
General Set (D_g)	Random 200 images from COCO val across diverse categories	Measuring general capability preservation
Text-Only Probe Set	50 text-only questions about the forgotten concept (no images)	Measuring language modality leakage
Adversarial Probe Set	50 images with the key identifying feature masked/blurred/altered	Measuring cross-cue generalization after unlearning

4.2 Primary Evaluation Metrics

Metric 1: Forget Accuracy (FA)

The VQA accuracy of the model on the Forget Set for target-concept questions. For base model this should be high (>70%). For a well-unlearned model this should drop significantly (target <30%). Measured as exact-match or semantic similarity of generated answers.

Metric 2: Retain Accuracy (RA)

The VQA accuracy on the Retain Set. Should remain close to the base model's score (within 10-15% drop). A method that drops RA by >20% is considered to have catastrophic forgetting of related concepts.

Metric 3: General Utility Score (GU)

Accuracy across the General Set on diverse tasks (captioning quality via METEOR/ROUGE-L, VQA accuracy on COCO-VQA format questions). Ensures the intervention doesn't break the model globally.

Metric 4: Language Leakage Score (LL)

The accuracy of the model on text-only questions about the forgotten concept (no image provided). For example: 'What are zebras known for visually?' If the vision SAE is unlearned but the LM is intact, this score should remain HIGH — showing that language knowledge is retained while visual knowledge is erased. This is the novel measurement your project contributes.

Language Leakage is a key contribution: A high LL score after vision-only unlearning demonstrates that the LM still 'knows' the concept via text, but the visual pathway is severed. Conversely, if LM is also unlearned, LL should drop. The difference LL(vision-only) - LL(both) quantifies how much the language model 'remembers' the concept.

Metric 5: Membership Inference Attack (MIA) Score

Using Min-K% Probability method from the MUSE benchmark: probe whether the model's loss profile reveals it was ever 'trained' on the forget concept. For SAE-steered models (no gradient updates), this is particularly interesting — since no weights changed, the MIA might still succeed, exposing a fundamental limitation of inference-time-only unlearning.

Metric 6: Adversarial Robustness (AR)

Accuracy on the Adversarial Probe Set (blurred/masked images). If unlearning truly removed the concept representation, performance on partially-cued images should be low. If the model still recognizes 'zebra' from a half-blurred zebra image, unlearning is shallow.

4.3 Evaluation Summary Table Template

For each trained variant, fill this table (this is your final results table for any paper/report):

Method	FA ↓	RA ↑	GU ↑	LL	MIA ↓	AR ↓
Base Model (no unlearning)	~80%	~80%	~80%	~85%	High	~75%
Vision-Only Zero Ablation (SAE)	TBD	TBD	TBD	TBD	TBD	TBD

Vision-Only Steering (SAE) [MAIN]	TBD	TBD	TBD	TBD	TBD	TBD
Language-Only Steering (SAE)	TBD	TBD	TBD	TBD	TBD	TBD
Both Modalities Gradient (MultiDelete)	TBD	TBD	TBD	TBD	TBD	TBD
Both Modalities SAE Steering	TBD	TBD	TBD	TBD	TBD	TBD

5. Unlearning Methods

The project implements and compares the following methods, applied to each aspect. For each method, three variants exist: vision-only, language-only, and both-modalities.

Method 1: Zero-Ablation (Baseline)

What it does: Identifies SAE features most strongly activating for the target concept and sets their activation to exactly zero at inference time via forward-hook injection.

Expected outcome: Known to be damaging from LLM unlearning literature (Farrell et al., 2024). Should cause high FA reduction but also catastrophic utility degradation. This is your 'negative control' to show the problem with naive ablation.

Implementation: Register a forward hook on the SAE encoder output. For each image, if any of the target feature indices activate above threshold, clamp them to 0. No weight updates needed.

Compute cost: Essentially free — no training, only inference with hooks. ~1 hour Kaggle time for evaluation.

Method 2: SAE Feature Steering — Negative Clamping (Main Method)

What it does: Instead of zeroing features, clamps them to a strong negative value (e.g., -10 * mean activation). This actively pushes the representation away from the concept direction rather than merely silencing it.

Why better than zero-ablation: Zero ablation removes the signal but leaves the representation in an ambiguous null state. Negative clamping actively redirects the representation, producing more coherent downstream behavior.

Variants to test: (a) Clamp to -k*mean for $k \in \{5, 10, 20\}$, (b) Clamp only when activation exceeds 2σ above mean (conditional clamping), (c) Steer toward a randomly chosen 'neutral' feature direction.

Implementation: Forward hook on SAE activations. The hook function: if feature_idx in target_features and act > threshold: act = -scale * baseline_mean.

Compute cost: ~2-3 hours Kaggle time for ablation over scale values + full evaluation.

Method 3: SAE-Guided Weight Projection (Permanent Steering)

What it does: Uses the SAE decoder directions for the target features to project those directions out of the CLIP encoder's weight matrices. This permanently modifies weights so no hook is needed at inference — and crucially, this is not reversible by a simple prompt or MIA.

Mathematical form: For each target feature direction d (SAE decoder column), for each relevant weight matrix W: W' = W - (W @ d_norm) * d_norm^T. This projects W orthogonal to d, suppressing the feature at the model level.

Why important: Inference-time hooks don't change model weights, so MIA can still detect the concept in the model. Weight projection creates true weight-level modification while remaining interpretability-guided.

Compute cost: ~1-2 hours Kaggle time (no training, just matrix operations + evaluation).

Method 4: Gradient-Based Unlearning (Baseline Comparison)

What it does: Uses Negative Preference Optimization (NPO) or Gradient Ascent on the forget set, applied to either the vision encoder weights, the LM weights, or both (MultiDelete-style).

Why include: This is the established state-of-the-art. SAE methods need to be compared against gradient methods to show their advantages (interpretability, controllability, no training data needed) and disadvantages (potentially shallower forgetting).

Implementation: For vision-only: freeze LM + projector, run gradient ascent on CLIP encoder weights with forget set. For LM-only: freeze CLIP + projector, run gradient ascent on Vicuna weights. For both: unfreeze all, run jointly. Use 3-5 epochs with early stopping on retain accuracy.

Compute cost: This is the most compute-intensive method: ~6-10 hours Kaggle per variant (vision-only, LM-only, both). Plan 3 Kaggle accounts for the 3 variants.

6. Independent Task Breakdown

All tasks are designed to be independently executable. Each person doing a task should read only their task section (and the global assumptions box below). Models are saved to HuggingFace Hub, evaluation code is shared via GitHub, results are logged to a shared Google Sheet.

GLOBAL ASSUMPTIONS FOR ALL TASKS: • Primary model: llava-hf/llava-1.5-7b-hf (HuggingFace) • Vision SAE: ExplainableML/sae-for-vlm (clone from GitHub, use pretrained CLIP ViT-L/14 checkpoint) • Language SAE: EvolvingLMMs-Lab/multimodal-sae (clone from GitHub) • Primary forget aspect: Zebra / stripe texture (ImageNet class n02391049) • All code goes to GitHub repo: [your-org]/vlm-unlearning-sae • All trained models/modified checkpoints go to HuggingFace: [your-org]/vlm-unlearning-sae-checkpoints • All evaluation results go to shared Google Sheet: 'VLM Unlearning Results' • Kaggle environment: Python 3.11, CUDA 12, 2x T4 GPUs (16GB each) • Load model in fp16 (not 4bit) for training tasks; 4bit is acceptable for evaluation-only tasks • Always set seed=42 for reproducibility

TASK T-01: Environment Setup & SAE Feature Discovery

Assigned to: 1 person | **Estimated Time:** 4-6 hours

Goal

Set up the complete project environment on Kaggle, load LLaVA-1.5-7B and the CLIP SAE, and identify which specific SAE features correspond to the target concepts (zebra stripes, redness, fire truck). Output a JSON file mapping each concept to its top-K activating SAE feature indices.

Step-by-Step Instructions

1. Create a new Kaggle notebook. Set accelerator to 'GPU T4 x2'. Enable internet access.
2. Install dependencies in the first cell: !pip install transformers accelerate bitsandbytes pillow sentencepiece !git clone https://github.com/ExplainableML/sae-for-vlm.git !pip install -r sae-for-vlm/requirements.txt !git clone https://github.com/EvolvingLMMs-Lab/multimodal-sae.git
3. Load LLaVA-1.5-7B in fp16: model = LlavaForConditionalGeneration.from_pretrained('llava-hf/llava-1.5-7b-hf', torch_dtype=torch.float16).cuda() processor = AutoProcessor.from_pretrained('llava-hf/llava-1.5-7b-hf')
4. Load the pretrained CLIP ViT-L/14 SAE from sae-for-vlm. Follow the repo's README to load the checkpoint. Register a forward hook on the SAE encoder to capture feature activations for each image.
5. Download a small batch of concept images: ~30 zebra images from ImageNet (class n02391049), ~30 horse images (n02374451), ~30 fire truck images (search COCO or use HuggingFace datasets). Also download ~30 random non-concept images as baseline.
6. Run all images through CLIP → SAE. For each image, record which SAE features activate (activation value > 0). Compute, for each feature, the mean activation on concept images minus mean activation on baseline images. This is the 'concept selectivity score'.
7. Rank features by selectivity score. Select top-20 features for each concept. Save results as: concept_features.json = {'zebra': [feat_idx_1, ...], 'redness': [...], 'fire_truck': [...]}
8. Verify by visual inspection: For top-5 features of zebra, show the 16 most activating ImageNet images (use activations.csv from sae-for-vlm benchmark data). These should visually correspond to zebra/stripes.
9. Upload concept_features.json and the verification visualizations to the GitHub repo under /results/feature_discovery/.

Deliverables

- concept_features.json (GitHub)
- feature_verification_visuals.png — grid of top-activating images for each discovered feature (GitHub)
- Kaggle notebook link (set to public) in GitHub README
- Note in Google Sheet: which layer of CLIP the SAE is applied to, total number of features in SAE, selectivity scores for top features

TASK T-02: Dataset Curation & Baseline Evaluation

Assigned to: 1 person | **Estimated Time:** 5-7 hours

Goal

Build the full evaluation dataset (Forget Set, Retain Set, General Set, Text-Only Probe Set, Adversarial Probe Set) for the zebra aspect, and run the base LLaVA-1.5-7B model to establish all baseline scores. These baseline numbers are the reference point for every subsequent task.

Step-by-Step Instructions

1. Load the datasets. Use HuggingFace datasets library: Forget Set: ImageNet validation, zebra class (n02391049). Take first 80 images. Retain Set: ImageNet validation, horse (n02374451) + donkey (n02389026). Take 100 images total. General Set: COCO validation, 200 randomly sampled images across all categories.
2. For each Forget Set and Retain Set image, create 3 VQA question-answer pairs: • 'What animal is shown in this image?' → ground truth: 'zebra' (Forget) or 'horse' (Retain) • 'Describe the pattern/texture of the animal.' → ground truth: 'black and white stripes' (Forget) or 'brown/plain coat' (Retain) • 'Is this a zebra or a horse?' → ground truth: 'zebra' or 'horse'
3. Create the Adversarial Probe Set (25 images). Take zebra Forget Set images and apply two treatments using PIL: Treatment A: Gaussian blur (radius=15) applied to the animal body region (manually crop bounding box from COCO annotations) Treatment B: Greyscale conversion of just the animal body (removing color cues) Save both versions alongside original.
4. Create the Text-Only Probe Set: 25 text questions with no image input: 'What does a zebra look like?', 'What is the most distinctive visual feature of a zebra?', 'What color pattern do zebras have?', 'Can you tell the difference between a zebra and a horse just by looking?' For each question, the expected answer references 'stripes' or 'black and white'.
5. Load LLaVA-1.5-7B in 4bit for this evaluation task: model = LlavaForConditionalGeneration.from_pretrained('llava-hf/llava-1.5-7b-hf', load_in_4bit=True)
6. Run evaluation loop. For each image+question, generate model answer (max_new_tokens=100). Compute: • FA: % of Forget Set questions answered correctly (string match 'zebra' in response) • RA: % of Retain Set questions answered correctly • LL: % of Text-Only Probe questions that mention 'stripes' or 'black and white' • AR: % of Adversarial Probe questions answered correctly
7. Save all model responses to baseline_responses.json. Record all scores in Google Sheet row 'Base Model (no unlearning)'.
8. Upload dataset files (as HuggingFace dataset) and baseline_responses.json to GitHub.

Deliverables

- HuggingFace dataset: [your-org]/vlm-unlearning-zebra-eval (Forget/Retain/General/TextOnly/Adversarial splits)
- baseline_responses.json (GitHub)

- Google Sheet row: Base Model scores for all 6 metrics

TASK T-03: Vision-Only Zero Ablation

Assigned to: 1 person | **Estimated Time:** 3-4 hours

Goal

Implement zero-ablation on the CLIP vision encoder using the SAE features identified in T-01. This is the 'naive' baseline expected to perform poorly — confirm or refute prior findings from text-only models in the multimodal setting.

Prerequisite

T-01 must be complete. You need concept_features.json from GitHub. You need the evaluation dataset from T-02.

Step-by-Step Instructions

1. Clone the GitHub repo. Load concept_features.json. The zebra features list is your target_feature_indices.
2. Load LLaVA-1.5-7B in fp16. Load the CLIP SAE (same as T-01).
3. Register a PyTorch forward hook on the SAE's post-activation layer (after the ReLU). The hook function: def zero_ablation_hook(module, input, output): output[:, target_feature_indices] = 0 return output This sets the activation of all target features to zero for every forward pass.
4. Test the hook works: Run a zebra image through the model with the hook active. The model response should change. Run a horse image — response should be minimally affected (since horse features differ from zebra features).
5. Run the full evaluation loop (same as T-02 step 6) with the hook active. Record FA, RA, GU, LL, AR scores.
6. Ablation experiment: Test with different numbers of ablated features (top-5, top-10, top-20 features). Record how FA and RA change with each setting.
7. Save modified model state (or just the hook configuration + feature indices) to HuggingFace as: [your-org]/vlm-unlearning-sae-checkpoints/vision_zero_ablation_top20_zebra
8. Upload results to Google Sheet row 'Vision-Only Zero Ablation'. Save response JSON as zero_ablation_responses.json to GitHub.

Expected Findings

Based on prior literature: FA should drop but RA and GU should also degrade significantly. The model may start producing incoherent outputs. This is the expected 'bad result' that motivates Method 2 (steering). Document this carefully.

TASK T-04: Vision-Only SAE Steering (Main Method)

Assigned to: 1 person | **Estimated Time:** 6-8 hours

Goal

Implement negative clamping / direction steering on the CLIP vision encoder. This is the project's main contribution — a more surgical alternative to zero-ablation that steers features rather than silencing them.

Prerequisite

T-01 and T-02 must be complete. Ideally, T-03 is also complete so you can compare against it.

Step-by-Step Instructions

10. Load concept_features.json. For each target feature, compute the baseline mean activation on the Retain Set (50 retain images). This gives you baseline_mean[feature_idx]. Save this to baseline_stats.json.
11. Implement Method A — Negative Clamping hook: def neg_clamp_hook(module, input, output): for feat_idx in target_feature_indices: mask = output[:, feat_idx] > threshold # Only suppress when strongly activating output[:, feat_idx][mask] = -scale * baseline_mean[feat_idx] return output Test with scale=10, threshold=0.5.
12. Implement Method B — Decoder Direction Steering. Get the SAE decoder weight matrix D (shape: num_features × hidden_dim). The steering vector for feature i is D[i] (the decoder direction). Add this as a negative activation to the residual stream: steering_vector = -alpha * D[target_feature_indices].mean(0) # Aggregate direction The hook adds this vector to the encoder output for images containing the concept.
13. Implement Method C — Weight Projection (permanent). For each target feature direction d = D[feat_idx] / ||D[feat_idx]||: for each attention weight matrix W in CLIP: W = W - (W @ d.unsqueeze(1)) @ d.unsqueeze(0). Save the modified CLIP encoder weights.
14. Run full evaluation for all three variants. Record all 6 metrics for each. Test at multiple scale values for Method A (scale ∈ {5, 10, 20}).
15. Run the fun experiment: Adversarial Probe Set evaluation. Show blurred/masked zebra images to: (a) base model, (b) zero-ablation model, (c) steering model. Compare AR scores.
16. Save Method C (permanent weight projection) as the primary checkpoint to HuggingFace: [your-org]/vlm-unlearning-sae-checkpoints/vision_steering_zebra_best
17. Upload all response JSONs, plots of FA/RA vs scale, and checkpoint to GitHub + Google Sheet.

Key Metric to Watch

The Language Leakage score (LL) is the critical number here. After vision-only steering, the model should have low FA (can't identify zebra visually) but HIGH LL (still knows textually what a zebra looks like). This demonstrates modality-specific forgetting.

TASK T-05: Language-Only SAE Steering

Assigned to: 1 person | **Estimated Time:** 6-8 hours

Goal

Apply SAE steering to the language model component of LLaVA, leaving the visual encoder completely untouched. This produces the opposite of T-04: the model should still visually identify zebra (vision is intact) but should give non-informative text answers about it.

Prerequisite

T-01 and T-02 complete. The language SAE from EvolvingLMMs-Lab/multimodal-sae must be available.

Step-by-Step Instructions

18. Important first step: Check if the EvolvingLMMs-Lab multimodal-sae repo has pretrained checkpoints for LLaVA-1.5-7B or LLaVA-NeXT. If only LLaVA-NeXT is supported: either switch the base model for this task to LLaVA-NeXT (llava-hf/llava-v1.6-mistral-7b-hf) or train a small language SAE. Document your decision in the GitHub README under T-05.
19. If training a language SAE is needed: Use the training code from the multimodal-sae repo. Train on 5000 forward passes of the Retain Set + General Set through the LM layers (freeze everything except the SAE). This should take ~4-6 hours on Kaggle T4. Train SAE on a middle LM layer (layer 16 out of 32 is a reasonable choice).
20. Identify language SAE features corresponding to the zebra concept. Method: Run text 'A zebra is an animal with black and white stripes.' through the LM with the SAE. Identify features that activate strongly. Cross-reference with 'A horse is a brown animal.' to find zebra-specific language features.
21. Apply the same steering methods from T-04 (negative clamping, decoder direction, weight projection) but to the language SAE features instead of vision SAE features. The hook is now on the LM layer where the SAE is inserted.
22. Run full evaluation. Key expected finding: FA on image-based VQA may NOT drop (or drop less) because CLIP still encodes the zebra correctly. But LL (text-only answers) should drop — the LM can no longer articulate what a zebra is.
23. This is a key comparison to T-04. Document in detail: After vision-only steering, which questions does the model fail on? After LM-only steering, which questions does it fail on? They should fail on different question types.
24. Save checkpoint and all results to GitHub + HuggingFace + Google Sheet.

TASK T-06: Both-Modalities Gradient Unlearning (MultiDelete Baseline)

Assigned to: 1 person | **Estimated Time:** 8-12 hours

Goal

Implement gradient-based unlearning on both modalities jointly, serving as the 'strongest baseline' — the method with the most compute but no SAE interpretability. Compare against all SAE methods.

Prerequisite

T-02 complete (need the forget/retain dataset). This task runs independently of T-03, T-04, T-05.

Step-by-Step Instructions

9. Load LLaVA-1.5-7B in fp16. Unfreeze ALL parameters (CLIP encoder, MLP projector, and Vicuna LM).
10. Prepare the gradient ascent objective on the Forget Set: loss_forget = -model(images=forget_imgs, questions=forget_qs, answers=forget_ans).loss. This makes the model maximize loss (i.e., get worse) on forget set examples.
11. Prepare the gradient descent objective on the Retain Set: loss_retain = model(images=retain_imgs, questions=retain_qs, answers=retain_ans).loss. This preserves performance on retained knowledge.
12. Combined loss (Gradient Difference method): total_loss = -loss_forget + lambda * loss_retain with lambda=0.5. Use AdamW optimizer with lr=2e-5.
13. Train for up to 5 epochs. After each epoch, evaluate FA and RA. Implement early stopping: stop if RA drops more than 15% below baseline OR if FA drops below 25%. Save checkpoint at best FA/RA tradeoff.
14. Run full evaluation (all 6 metrics). Critically, measure LL: does the gradient-based method remove language knowledge of zebra as well? It likely should since gradients flow through both modalities.
15. Important: This task uses the most Kaggle compute (~8-12 hours). Use a dedicated Kaggle account. If running short of time, use Gradient Ascent only (no retain set constraint) as a simplified version — note this in results.
16. Save best checkpoint to HuggingFace: [your-org]/vlm-unlearning-sae-checkpoints/gradient_both_modalities_zebra. Upload all results.

TASK T-07: Both-Modalities SAE Steering

Assigned to: 1 person | **Estimated Time:** 5-7 hours

Goal

Apply SAE steering simultaneously to both the vision encoder and the language model. This is the SAE-equivalent of Task T-06 and allows direct comparison: does applying SAEs to both modalities approach the performance of gradient-based joint unlearning?

Prerequisite

T-04 and T-05 complete. You will reuse the hooks and feature indices from both.

Step-by-Step Instructions

25. Load the vision SAE features from T-04 (concept_features.json) and language SAE features from T-05. Combine both hooks into a single forward pass.
26. The model now has two hooks simultaneously: one on the CLIP SAE suppressing zebra visual features, one on the LM SAE suppressing zebra language features.
27. Run full evaluation. Key hypothesis: FA should be lower than either vision-only or LM-only (additive forgetting effect). LL should be lower than vision-only (language knowledge also suppressed).
28. Compare against T-06 (gradient both-modalities): Which achieves lower FA? Which better preserves RA and GU? This is a core finding of the paper.

29. Run the fun experiment: Show a blurred zebra image. Does both-modalities SAE unlearning cause lower AR than vision-only? It should, because even if some visual cue leaks through, the LM can no longer process it into a coherent zebra identification.
30. Save results to GitHub + HuggingFace + Google Sheet.

TASK T-08: Cross-Aspect Generalization & Knowledge Leakage Analysis

Assigned to: 1 person | **Estimated Time:** 4-6 hours

Goal

Using the best-performing model from T-04 (vision-only SAE steering), run extended experiments to characterize cross-aspect transfer and knowledge leakage. This is the analysis and fun-experiment task.

Step-by-Step Instructions

17. Load the best vision-only SAE checkpoint from T-04 (from HuggingFace).
18. Run the full fun experiment: Create 3 versions of each zebra Forget Set image: (a) Original image (b) Stripes blurred (Gaussian blur on animal body) (c) Animal body area replaced with uniform grey patch For each, ask 'What animal is this?' and 'What pattern does this animal have?'. Record responses for base model vs. unlearned model.
19. Cross-category generalization test: Show images of OTHER striped animals (tigers, barred owls, striped fish) to the unlearned model. Does unlearning 'zebra stripe' features also cause the model to fail on these? This would indicate the SAE feature was texture-specific (bad) or zebra-specific (good).
20. Text-image inconsistency experiment: Prompt the model with a text description of a zebra + a blank/white image and ask 'What animal am I describing?'. Compare base model vs. unlearned model. This probes whether language-side knowledge enables reconstruction of the forgotten visual concept.
21. Run aspect B (redness) if time allows: Use concept_features.json for redness, apply vision-only steering, and evaluate on red vs. non-red objects. This tests generalizability of your pipeline to a second aspect.
22. Compile findings into a clear analysis document: leakage_analysis.md on GitHub. Include the fun experiment results with image examples.

TASK T-09: Evaluation Script (Shared Codebase)

Assigned to: 1 person | **Estimated Time:** 4-5 hours

Goal

Write a single, reusable evaluation script that any task (T-03 through T-08) can call to generate all 6 metrics. This person should ideally be someone comfortable with Python/ML code and starts early (even before T-03).

Step-by-Step Instructions

9. Create evaluate.py in the GitHub repo. Function signature: def evaluate_model(model, processor, dataset_path, hook_fn=None, device='cuda'): # Returns dict: {'FA': float, 'RA': float, 'GU': float, 'LL': float, 'AR': float, 'MIA': float} The optional hook_fn parameter allows any task to inject their SAE hooks.
10. FA computation: Loop over Forget Set. For each image+question, generate response (max_new_tokens=50, greedy). Score as 1 if 'zebra' appears in response (case-insensitive), else 0. Average.
11. RA computation: Same but on Retain Set. Score as 1 if 'horse' appears in response (or correct retain label).
12. GU (General Utility) computation: On General Set, use ROUGE-L score between model response and a reference caption (use COCO captions as reference). Use the rouge_score library.
13. LL (Language Leakage) computation: Run text-only questions (no image — pass a blank white image). Score as 1 if response contains 'stripe' or 'black and white' or 'monochrome pattern'.
14. MIA computation: For each forget set image, compute model's cross-entropy loss. For each retain set image, compute the same. Fit a logistic regression classifier that predicts 'forget' vs. 'retain' from loss. AUC-ROC is the MIA score (0.5 = perfect unlearning, 1.0 = complete membership information retained).
15. AR computation: Run the adversarial probe set (blurred images). Use same FA scoring (does model still say 'zebra'?).
16. Write a results_to_sheet.py helper that formats the output dict as a CSV row for easy copy-paste to Google Sheet.
17. Test the script on the base model to confirm it matches T-02's manually computed baseline scores.

7. Compute & Time Planning

Task	Kaggle Account	Estimated Hours	Dependencies
T-01: Feature Discovery	Account 1	4-6h	None
T-02: Dataset & Baseline	Account 2	5-7h	None (parallel with T-01)
T-09: Evaluation Script	Any (no GPU needed)	4-5h	T-02 for dataset format
T-03: Vision Zero Ablation	Account 3	3-4h	T-01, T-02, T-09
T-04: Vision SAE Steering	Account 4	6-8h	T-01, T-02, T-09
T-05: Language SAE Steering	Account 5	6-8h	T-01, T-02, T-09

T-06: Gradient Both Modalities	Account 6	8-12h	T-02, T-09
T-07: Both Modalities SAE	Account 7	5-7h	T-04, T-05 complete
T-08: Leakage Analysis	Account 8	4-6h	T-04 complete

Total Kaggle GPU hours across all accounts: ~46-63 hours. With 30 hours per Kaggle free account, you need at minimum 2 accounts and ideally 4-6 separate accounts to run tasks in parallel. T-06 is the most expensive — if compute is tight, simplify to Gradient Ascent only (no retain set constraint). Tasks T-01, T-02, and T-09 can be done on a single account sequentially as the first 12-15 hours before parallelizing.

Parallelization Strategy

Phase 1 (Days 1-2): T-01, T-02, T-09 — run these first as all others depend on their outputs. T-01 and T-02 can be parallelized on separate accounts.

Phase 2 (Days 3-4): T-03, T-04, T-05, T-06 — these are all independent and can all run simultaneously on 4 separate accounts once Phase 1 is done.

Phase 3 (Days 5-6): T-07 (requires T-04+T-05), T-08 (requires T-04). Run these once Phase 2 checkpoints are uploaded to HuggingFace.

8. GitHub Repository Structure

```
vlm-unlearning-sae/ ├── README.md                    # Project overview, setup
instructions ├── requirements.txt                # All dependencies ├── evaluate.py
# T-09: shared evaluation script ├── results_to_sheet.py              # T-09: Google Sheet
helper ├── tasks/ │   ├── T01_feature_discovery.ipynb   # T-01 notebook │   ├──
T02_dataset_baseline.ipynb    # T-02 notebook │   ├── T03_zero_ablation.ipynb       # T-
03 notebook │   ├── T04_vision_steering.ipynb      # T-04 notebook │   ├──
T05_language_steering.ipynb   # T-05 notebook │   ├── T06_gradient_unlearning.ipynb  #
T-06 notebook │   ├── T07_both_modalities_sae.ipynb  # T-07 notebook │   └──
T08_leakage_analysis.ipynb    # T-08 notebook ├── results/ │   ├── feature_discovery/ │
│   ├── concept_features.json │   │   └── feature_verification_visuals.png │   ├──
baselines/ │   │   └── baseline_responses.json │   ├── T03_zero_ablation_responses.json
│   ├── T04_vision_steering_responses.json │   ├──
T05_language_steering_responses.json │   ├── T06_gradient_responses.json │   ├──
T07_both_sae_responses.json │   └── leakage_analysis.md └── data/    └──
dataset_readme.md            # Instructions to download datasets
```

9. Known Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Language SAE for LLaVA-1.5 not available	Medium	T-05 blocked	Switch to LLaVA-NeXT for T-05, or train a small 1-layer SAE in ~3h
SAE features for zebra not cleanly separable	Low	T-01 results messy	Use top-30 instead of top-20; also use MSAE as backup
Gradient unlearning (T-06) runs out of compute	Medium	T-06 incomplete	Use gradient ascent only (no retain constraint), 2-3 epochs max
Zero-ablation causes complete model collapse	Medium	T-03 unusable	Document as expected failure; test with top-5 features only
Weight projection (T-04 Method C) degrades GU badly	Low-Medium	Main method fails	Fall back to inference-time negative clamping as primary method
Kaggle session timeouts	High (for long tasks)	Work lost	Save checkpoints every epoch; use Kaggle dataset to persist outputs between sessions

10. Expected Findings & Paper Narrative

Based on prior literature, here is our anticipated narrative — which may be confirmed or refuted by experiments:

- Zero-ablation (T-03) will produce the worst FA-RA tradeoff: it will either fail to forget (if too few features ablated) or catastrophically damage utility (if too many). This confirms in the multimodal domain what was observed in LLMs.
- Vision-only SAE steering (T-04) will achieve meaningful forgetting on image-based VQA (FA drops) while preserving language knowledge (LL stays high). This demonstrates modality-specific unlearning for the first time with an SAE-based method.
- The Language Leakage finding will be the key contribution: a model with vision unlearning can no longer identify a zebra from its image, but can still write a paragraph about what a zebra looks like when asked in text. This gap between visual and textual knowledge is quantified for the first time.
- Language-only SAE steering (T-05) will show the inverse: the model can still see the zebra in the image (CLIP is unmodified) but cannot verbalize the concept clearly. This is a strange but scientifically interesting model state.
- Both-modalities SAE steering (T-07) will achieve lower FA than either modality alone, but will likely still underperform gradient-based joint unlearning (T-06) on raw FA metrics. However, it will better preserve GU and RA, showing the interpretability-utility tradeoff.
- The fun blurring experiment will show that after vision-only SAE unlearning, the model is 'confused' by partially occluded zebras — it may identify them as 'horse' or 'animal', showing the removed features were genuinely necessary for recognition, not just a shallow surface pattern.

This narrative sets up a clean, publishable story: SAEs enable interpretable, modality-specific unlearning in VLMs, revealing that concepts are not uniformly encoded across modalities, and that vision-only forgetting creates a novel model state with cross-modal knowledge asymmetry.

11. Key References & Resources

Paper / Resource	Relevance	Link
Pach et al. (NeurIPS 2025) — SAEs Learn Monosemantic Features in VLMs	Primary SAE for vision; shows SAE→LLaVA steering	github.com/ExplainableML/sae-for-vlm
Joseph et al. (arXiv 2504.08729) — Steering CLIP with SAEs	Alternative CLIP SAE with defense against typographic attacks	arXiv:2504.08729
EvolvingLMMs-Lab (ICCV 2025) — Multimodal SAE	SAE on LLaVA language side; T-05 resource	github.com/EvolvingLMMs-Lab/multimodal-sae
Zaigrajew et al. (ICML 2025) — MSAE Hierarchical SAE	Backup vision SAE; hierarchical features on CLIP	github.com/WolodjaZ/MSAE
MultiDelete (ECCV 2024)	Key baseline: multimodal gradient unlearning method	Search 'MultiDelete multimodal unlearning ECCV 2024'
Farrell et al. (2024) — SAEs Can Improve Unlearning	Motivates steering over ablation; LLM baseline	Search 'SAEs Can Improve Unlearning Farrell 2024'
MUSE Benchmark	Evaluation framework: 6 metrics for unlearning	muse-bench.github.io
TOFU Benchmark	TOFU forget quality metric (KS test on truth ratio)	arxiv.org/abs/2401.06121
LLaVA-1.5-7B HuggingFace	Primary model	huggingface.co/llava-hf/llava-1.5-7b-hf

This document is a living project specification. As experiments run and findings emerge, update the Google Sheet and push results to GitHub. If a method dramatically underperforms or overperforms expectations, revisit the task design with the team before proceeding to dependent tasks. Good luck!